TP N°1 : Introduction à JADE, Installation et Premier Agent

Nom : **GOUNGOU Cédric Hamel**

Année Universitaire 2025-2026

1 Introduction

L'objectif de ce TP est d'installer le framework JADE, configurer l'IDE IntelliJ IDEA et créer un premier agent autonome. Ce travail permet de comprendre le fonctionnement des systèmes multi-agents (MAS) et leur exécution sur une plateforme dédiée.

2 Ajout des bibliothèques dans IntelliJ (Build Path équivalent)

Dans IntelliJ IDEA, l'ajout des JAR se fait via :
— `File > Project Structure > Libraries`
— Ajout des fichiers `jade.jar` et `commons-codec.jar`

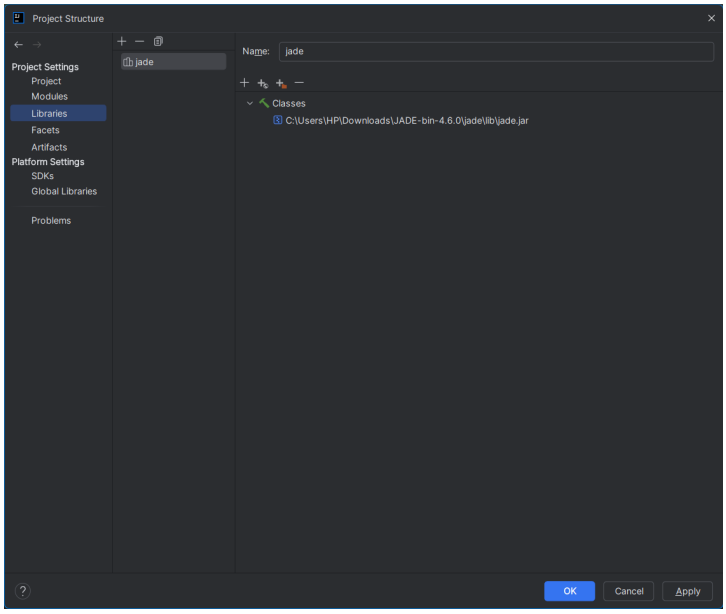

FIGURE 1 – Ajout de jade.jar et commons-codec.jar dans les bibliothèques du projet (IntelliJ)

3 Développement de l'Agent HelloAgent

3.1 Code source

```
package eu.uemf.agents;

import jade.core.Agent;

public class HelloAgent extends Agent {

    @Override
```

```
    protected void setup() {
        System.out.println("Hello World! I am a JADE Agent.");

        // Nom local de l'agent
        System.out.println("My local name is " + getAID().getLocalName());

        // Identifiant global unique (GUID)
        System.out.println("My GUID is " + getAID().getName());
    }
}
```

3.2 Explications techniques

- La classe `HelloAgent` hérite de la classe `Agent` fournie par JADE.
- La méthode `setup()` représente le point d'entrée de l'agent (équivalent à `main`).
- Les méthodes `getAID().getLocalName()` et `getName()` identifient l'agent localement et globalement.

4 Configuration du Run (IntelliJ)

Pour exécuter un agent JADE, on configure une exécution avec :

- **Main Class** : `jade.Boot`
- **Program Arguments** :

```
-gui -local-host 127.0.0.1 -agents myAgent:eu.uemf.agents.HelloAgent
```

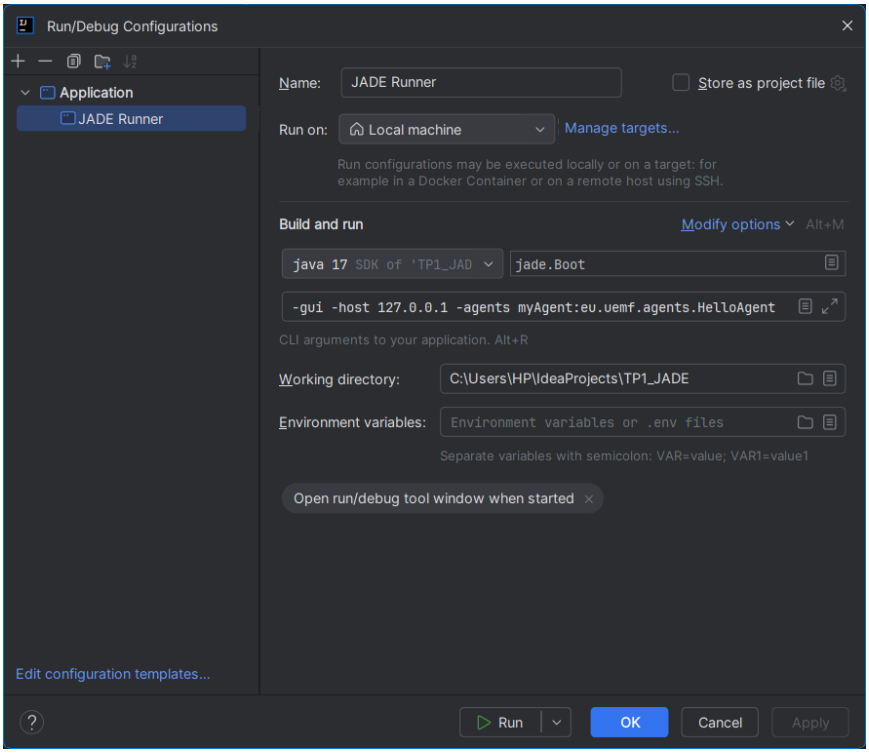

FIGURE 2 – Configuration du Run dans IntelliJ (classe jade.Boot et arguments)

5 Exécution et Résultats

5.1 Lancement de la plateforme JADE

Après configuration, l'exécution lance :

— La console Java affichant les messages de l'agent
— L'interface graphique JADE (RMA)

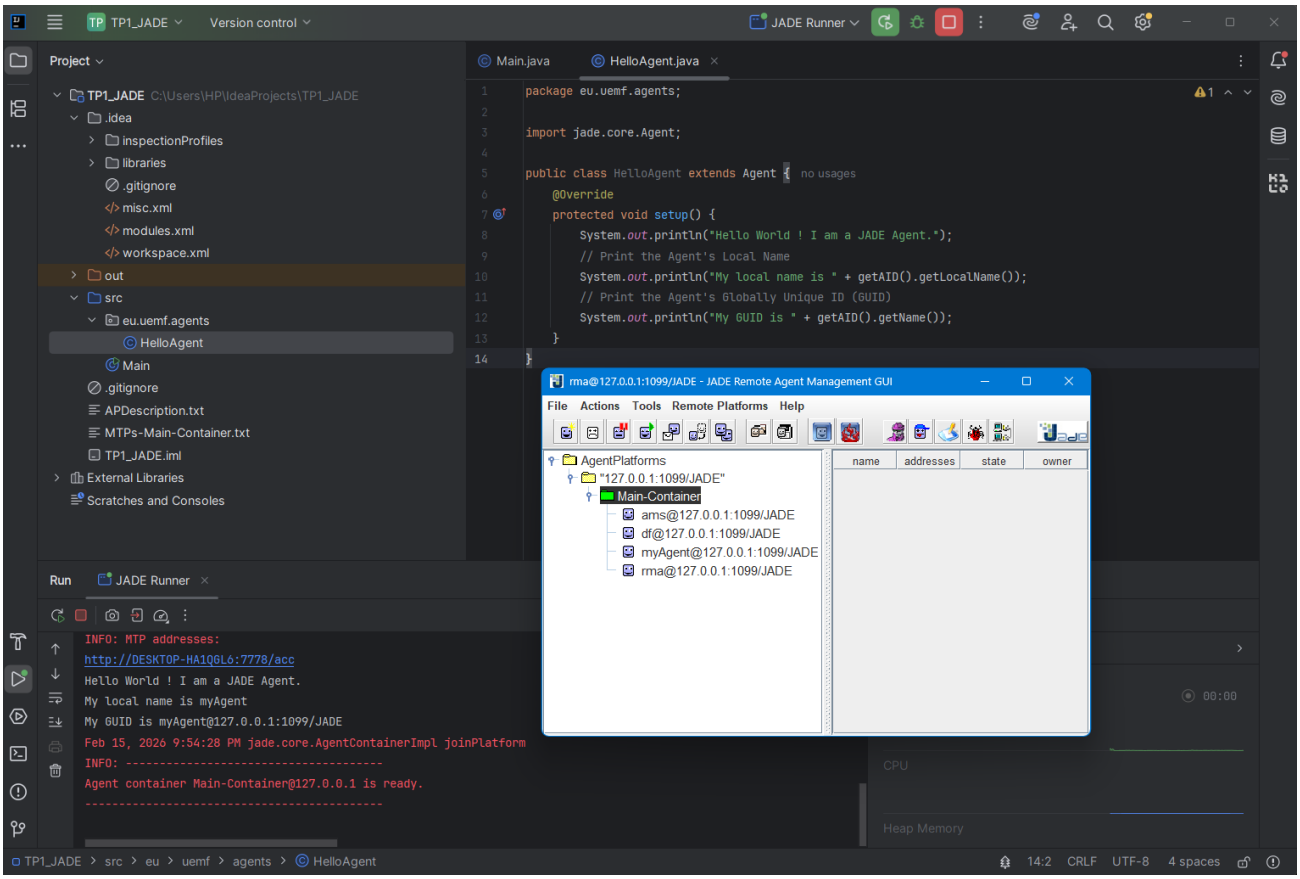

FIGURE 3 – Exécution de la plateforme JADE : console et interface GUI

5.2 Analyse de la console

La console affiche :
— Le message de démarrage de l'agent
— Le nom local de l'agent
— Son identifiant global unique (GUID)

5.3 Analyse de l'interface GUI (RMA)

L'interface RMA montre :
— **ams** : Agent Management System
— **df** : Directory Facilitator
— **myAgent** : notre agent créé

6 Partie II : Implémentation avec SPADE (Python)

6.1 Configuration de l'environnement

IDE utilisé : PyCharm.
Gestionnaire de paquets : Utilisation de `pip` dans un environnement virtuel nommé `tp1_mas_venv`.
Installation :

```
pip install spade spade_bdi
```

Cette installation permet de supporter la création d'agents SPADE ainsi que le modèle de raisonnement BDI (Belief-Desire-Intention).

6.2 Protocole de Communication (XMPP)

Contrairement à JADE, SPADE nécessite l'utilisation d'un serveur de messagerie XMPP pour assurer la communication entre agents. Pour ce TP, nous avons utilisé le serveur public :

`jabber.hot-chilli.net`

Différence notable : Le protocole XMPP (messagerie instantanée) ne doit pas être confondu avec XAMPP, qui est un serveur Web utilisé pour des applications PHP/MySQL.

6.3 Développement et Communication

Deux agents ont été implémentés :
— **SenderAgent** : agent émetteur
— **ReceiverAgent** : agent récepteur
Le code repose sur :
— La bibliothèque `asyncio` pour la gestion asynchrone
— La classe `Message` pour l'envoi et la réception des données

6.4 Analyse du résultat obtenu

Lors de l'exécution du programme, les observations suivantes ont été faites dans la console :
— L'agent récepteur se connecte au serveur XMPP et se met en attente de messages.
— Le serveur XMPP envoie automatiquement un message de bienvenue.
— L'agent émetteur envoie ensuite le message :

`"Salut ! C'est mon premier message SPADE."`

Observation technique : Le récepteur a intercepté en premier le message automatique envoyé par le serveur XMPP. Le message provenant de l'agent émetteur a ensuite provoqué l'alerte `No behaviour matched` car le comportement `OneShotBehaviour` était déjà terminé au moment de la réception.

7 Comparaison JADE vs SPADE

Critère	JADE	SPADE
Langage	Java	Python
Communication	Interne (Main Container)	Externe (Serveur XMPP)
Identifiant	AID (nom@host)	JID (user@serveur ex : https ://jabber.hot-chilli.net/forms/crea
Parallélisme	Threads Java	Asynchronisme (async/await)

8 Conclusion Générale

Ce TP a permis de comprendre les fondamentaux des Systèmes Multi-Agents (SMA). JADE se révèle robuste et adapté pour des infrastructures locales fixes avec une gestion interne des communications entre agents. En revanche, SPADE offre une plus grande flexibilité grâce à l'utilisation de Python et du protocole XMPP, ce qui facilite l'intégration future de bibliothèques d'Intelligence Artificielle telles que `torch` ou `scikit-learn`, déjà installées dans notre environnement de développement.